SecureVault

A technical overview of how the password manager works

Temidayo Peters · Version 3.1.2 of the application · September 2026

SecureVault is a local, offline password manager written in C# with Windows Forms on .NET 8. It keeps a person's website logins in one encrypted file on their own computer, unlocked by a single master password. Nothing is sent anywhere. It was built as A-Level Computer Science coursework and is published at github.com/Temi-Peters/SecureVault, with the full coursework report alongside the code.

This document explains what the application actually does, step by step, and where each step lives in the source. It is written from the code rather than from the original design, so it also records the limitations found on reading the code back, including one serious flaw in how the encryption key was stored, which was fixed in Version 3.1 and is kept here as the most instructive part of the project. The two point releases since, 3.1.1 and 3.1.2, fixed a frozen interface and a data location problem, and both are recorded in section 6. Everything stated here can be checked against the named file and method.

1. The shape of the application

There are five windows and four helper classes. The windows handle what the user sees; the helpers hold the logic that matters for security, kept apart from the interface code so it can be read and tested on its own.

File	Role
`Program.cs`	Entry point. Creates the data folder, shows the login window, and only if login succeeds opens the main vault window, passing it the derived encryption key.
`LoginForm.cs`	Creates the master password on first run. Verifies it on every run after that. Applies the lockout.
`MainForm.cs`	The vault. Add, view and delete entries. Reads and writes the encrypted vault file.
`ViewPasswordForm.cs`	Shows one entry. Password masked by default, timed reveal, copy to clipboard with automatic clearing.
`ConfirmPasswordForm.cs`	Asks for the master password again before an entry can be viewed.
`ChangePasswordForm.cs`	Changes the master password after verifying the old one.
`PasswordUtils.cs`	All the cryptography: salt generation, PBKDF2, AES-256 encrypt and decrypt, the constant-time comparison, and reading and writing master.dat.
`LockoutManager.cs`	The exponential backoff after failed logins, persisted to lockout.dat.
`SecurityPolicy.cs`	The master password complexity rules.
`DataPaths.cs`	The one place that decides where the three data files live, and the first-run move of a vault left beside the executable by an earlier version.

2. The pipeline, end to end

Everything below follows the order a user experiences it. The names in brackets are the methods involved.

Step 1. First run: creating the master password

When the application starts, `LoginForm` checks whether `master.dat` exists (`PasswordUtils.LoadMaster`). If it does not, whatever the user types becomes the new master

password, provided it passes the policy in `SecurityPolicy.ValidatePasswordComplexity`: at least 10 characters, at least one uppercase letter, one lowercase letter, one digit and one symbol, and it must not contain *password*, *12345* or *qwerty*. If the policy rejects it, the exact rule that failed is shown and nothing is saved.

If it passes, three things happen in `PasswordUtils`:

- **A salt is generated** (`GenerateSalt`): 16 random bytes from the operating system's cryptographic random number generator. The salt is not secret. Its job is to make this vault's derived values unique, so that two people who happen to choose the same master password get completely different results, and so that nobody can precompute a table of answers in advance.
- **The password is run through PBKDF2** (`DeriveEncryptionKey`): PBKDF2 with HMAC-SHA256, the salt, 100,000 iterations, producing the 32-byte AES key. Section 3 explains what the iterations are for.
- **A verifier is computed from the key** (`ComputeVerifier`): the SHA-256 of those 32 bytes. This, and not the key, is what goes on disk.
- **Salt, iteration count and verifier are written to master.dat** (`SaveMaster`). Neither the master password nor the key is ever written anywhere.

The user is then asked to log in with the password they just created, rather than being let straight in.

Step 2. Every later run: verifying the master password

Before anything else, `LoginForm` asks `LockoutManager.GetRemainingLockoutSeconds` whether a lockout is in force. If it is, the attempt is refused with the remaining wait shown, and the password is not even checked.

Otherwise the salt, iteration count and verifier are read back from `master.dat`. The typed password is run through PBKDF2 to produce a candidate key, the candidate's SHA-256 is computed, and that is compared with the stored verifier using `CryptographicOperations.FixedTimeEquals` (`VerifyAndDeriveKey`). The comparison deliberately takes the same time whether the mismatch is in the first byte or the last, so timing cannot reveal how close a guess was.

On a wrong password, `LockoutManager.RegisterFailedAttempt` increments a failure count, records the time, and saves both to `lockout.dat`. The wait before the next attempt is 5 seconds multiplied by 2 to the power of (failures minus 1): 5, 10, 20, 40, 80, 160, then capped at 300 seconds. Because it is saved to disk, closing and reopening the application does not reset it.

On a correct password the failure count is reset to zero, and the candidate key, which has just been proven correct, becomes the AES key for the vault. It exists only in memory and is handed to `MainForm` when it opens. PBKDF2 runs once per login, not twice.

If `master.dat` is in the older layout from before 3.1, the login still works, and the file is rewritten in the new layout the moment the password is proven, so old vaults migrate themselves.

How this step used to work, and why it changed. Before 3.1, the value stored to verify the password was the raw PBKDF2 output, and the AES key was the raw PBKDF2 output of the same inputs. PBKDF2 is deterministic, so they were identical bytes: the file written to check a login *was* the vault key, and anyone holding both files could decrypt the vault without guessing anything. Storing a hash of the key instead means the file now proves a password is right without containing the key. Section 6 has the detail.

Step 3. Adding an entry

`MainForm.btnAdd_Click` takes the site, username and password from the three text boxes. The site must match a URL pattern (`IsSiteValid`), and site and password cannot be blank; username is optional. The three are joined into one string, `site | username | password`, and added to an in-memory list.

Then `SavePasswords` rewrites the whole vault file. Each entry is encrypted on its own with `PasswordUtils.EncryptString`: AES-256 in CBC mode, using the key from login, with a fresh random 16-byte initialisation vector generated for that entry. The IV is written in front of the ciphertext. Encrypting entries separately means a corrupted entry does not take the others with it, and the fresh IV means two identical entries produce different bytes on disk.

Step 4. Opening the vault

`MainForm.LoadPasswords` runs when the vault window opens. It reads the entry count, then each length-prefixed ciphertext, and decrypts each with `DecryptBytes`, which strips the IV off the front and uses it with the key. Only the site name of each entry is shown in the list on screen; usernames and passwords stay in memory. If the file is truncated or unreadable the user is warned and the file is reset rather than the application crashing.

Step 5. Viewing an entry

Clicking View does not show anything yet. It opens `ConfirmPasswordForm`, which asks for the master password again and checks it against `master.dat` in the same way as login. Only on success does `ViewPasswordForm` open, and it does several things to limit exposure:

- The password field is read-only, keyboard shortcuts are disabled on it, and it is masked by default.
- **Show** reveals the password and starts a 10-second timer. When it fires, the field masks itself again. Clicking Show again restarts the timer.
- If the window loses focus for any reason, the field re-masks immediately (`OnDeactivate`).
- **Copy Password** puts the password on the clipboard and starts a 15-second timer. When it fires, the clipboard is cleared, but only if it still contains what was copied. If the user copied something else in the meantime, that is left alone.
- **Close** blanks the field and stops both timers before the window shuts (`SecureCleanup`).

Step 6. Deleting an entry and changing the master password

Delete asks for confirmation, removes the entry from the in-memory list, and rewrites the vault file. Change Master Password verifies the old password, applies the policy to the new one, generates a new salt, runs PBKDF2 on the new password, and overwrites `master.dat`. Because the entries are already decrypted in memory, `MainForm` then re-encrypts and saves the whole vault under the new key in the same operation, so nothing is left encrypted under the old one.

3. What the 100,000 iterations are actually doing

A master password is a short string a person can remember. An AES-256 key is 32 bytes of data that should look random. Key derivation is the function that turns one into the other, and the design question is how expensive that function should be to run.

A single SHA-256 would produce 32 bytes, but SHA-256 is built to be fast: a high-end consumer graphics card computes on the order of 20 billion of them per second. If the key were a single hash of the password, an attacker holding the vault could try candidate passwords at that rate.

PBKDF2 feeds the hash back into itself 100,000 times. Each iteration is an HMAC, which is two SHA-256 operations, so deriving one key costs roughly 200,000 hash operations. The legitimate user pays that once at login, around a tenth of a second. An attacker pays it on every guess. The same graphics card drops from about 20 billion guesses per second to about 100,000.

What that does and does not buy. PBKDF2 multiplies the strength the password already has. It does not add any. Two worked examples, comparing 20 billion guesses per second without PBKDF2 against 100,000 with it:

Master password	Realistic entropy	Without PBKDF2	With 100,000 iterations
A human-chosen password that scrapes past the policy	about 35 bits	under 2 seconds	about 4 days
10 genuinely random printable characters	about 65 bits	about 95 years on one card	millions of years

Same algorithm, same iteration count, and the outcome is decided largely by the password. That is the honest summary: the iterations multiply the time by the same factor either way, so they cannot rescue a weak password, and for a strong one they are what turns a determined attacker's decades into something no amount of hardware makes practical.

For context, OWASP's current recommendation for PBKDF2-HMAC-SHA256 is 600,000 iterations, six times the figure used here. It was a defensible number when the project was written and the general guidance is that it should rise over time, which is why the count is stored in master.dat rather than fixed in the code: new vaults can be created at a higher count without breaking old ones.

PBKDF2 was chosen over Argon2id, which is generally preferred today, for three stated reasons: it is NIST-approved with FIPS 140-2 validated implementations, it is built into .NET with no external dependency, and it gave an acceptable login time. OWASP's own guidance names FIPS validation as the case where PBKDF2 should be preferred, so the choice sits inside that carve-out.

4. The three files on disk

All three live in the user's local application data folder, `%LOCALAPPDATA%\SecureVault`, which on a typical machine is `C:\Users\<name>\AppData\Local\SecureVault`. `DataPaths` builds the three full paths from that one folder, and `Program` creates the folder before any window opens. The files are created the first time they are needed, and all three are plain binary, written with a `BinaryWriter`. Integers are little-endian 32-bit unless stated.

File	Layout	Size
`master.dat`	int32 format marker (-2) | int32 iteration count | int32 salt length (16) | 16-byte salt | int32 verifier length (32) | 32-byte SHA-256 of the key	64 bytes
`passwords.dat`	int32 entry count, then for each entry: int32 length | 16-byte IV | AES-256-CBC ciphertext (a multiple of 16 bytes)	4 + per entry
`lockout.dat`	int32 failed attempt count | int64 timestamp of the last failure (.NET ticks, UTC)	12 bytes

The format marker is negative so it can never be confused with the salt length that begins a pre-3.1 file, which is how the loader tells the two layouts apart. The hash algorithm and AES mode are still fixed in the code; the iteration count is the one parameter likely to need raising, and it is the one now stored.

Up to 3.1.1 the files were opened by bare name, `"master.dat"`, which means whatever folder the process happened to start in. That is the folder you are standing in when launched from a terminal, and something else entirely when launched by double-clicking, sometimes a folder the user cannot write to, in which case the first save failed and the application appeared to hang. Keeping the files in the application data folder removes that dependence on how the application was started. On its first run, 3.1.2 looks beside the executable and in the working directory for a vault left by an earlier version, copies the three files into the new folder, and removes the originals if it is allowed to.

5. What it defends against, and what it does not

Defends against, as intended:

- **Someone guessing the master password at the login screen.** The lockout doubles the wait on every failure and survives a restart. Seven wrong attempts cost about ten minutes of waiting in total, and every attempt after that costs five minutes more.
- **Shoulder surfing.** Passwords are masked by default, revealed only briefly, and re-masked the moment the window loses focus.
- **Clipboard scraping after a copy.** The clipboard is cleared 15 seconds after a password is copied.
- **Someone who copies both files.** master.dat holds a hash of the key, not the key, so the attacker's only route to the vault is guessing the master password through PBKDF2 at 100,000 iterations per guess. Section 3 sets out what that costs.
- **Reading the vault file on its own.** Without the salt from master.dat, an attacker holding only passwords.dat cannot even begin guessing.

Does not defend against:

- **A weak master password.** PBKDF2 multiplies the cost of each guess; it cannot make a guessable password unguessable. The policy sets a floor, and the floor is not high.
- **Malware running on the same computer.** Anything with access to the process can read the decrypted entries from memory, watch the keyboard for the master password, or read the clipboard inside the 15-second window. No local password manager defends against this; it is outside what the design attempts.
- **Memory inspection after use.** Decrypted passwords are .NET strings, which cannot be wiped and remain in memory until garbage collection.

6. Known limitations, and what each release fixed

All of these were found by reading the code back after it was written, or by running the built application somewhere other than the machine it was written on. The first table is what is still open. The rest is what was found and fixed, release by release, kept because the flaws and their fixes are the most useful thing in this document.

Still open

Limitation	Where	Detail and fix
1. A pipe character corrupts an entry.	`MainForm.cs`	Entries are stored as site | username | password and split on the pipe when read, so a password containing one is cut short when viewed. **Fix:** a structured format such as length-prefixed fields, or escaping the delimiter.
2. Secrets live in strings.	`ViewPasswordForm.cs`, `MainForm.cs`	.NET strings are immutable and cannot be zeroed. **Fix:** hold decrypted values in char or byte arrays and clear them explicitly when done.
3. The iteration count is below current guidance.	`PasswordUtils.cs`	100,000 against OWASP's 600,000. Because the count is now stored per vault, raising DefaultIterations affects new vaults only. **Fix:** raise the default, and re-derive existing vaults at the higher count on a successful login.

Fixed in 3.1

Was	Where	Detail
The stored verifier was the encryption key.	`PasswordUtils.cs`, `LoginForm.cs`	HashPassword and DeriveEncryptionKey were the same PBKDF2 computation, so the 32 bytes in master.dat were the AES key. An attacker with both files could decrypt the vault with no guessing. **Fixed:** master.dat now stores SHA-256 of the key. Pre-3.1 files are recognised by their layout and rewritten on the next successful login.
Changing the master password orphaned the vault.	`ChangePasswordForm.cs`, `MainForm.cs`	A new salt and verifier were written but passwords.dat stayed encrypted under the old key. **Fixed:** ChangePasswordForm hands the new key back and MainForm re-encrypts the vault under it in the same operation.
The iteration count was not stored.	`PasswordUtils.cs`	100,000 existed only as a default parameter, so it could never be raised without breaking every existing vault. **Fixed:** the count is written to master.dat and read back at login.
The constant-time comparison was hand-written.	`PasswordUtils.cs`	A leftover from .NET Framework 4.7.2, which lacked one. **Fixed:** CryptographicOperations.FixedTimeEquals.

Fixed in 3.1.1

Was	Where	Detail
The unused session key rotation froze the interface.	`MainForm.cs`	A Windows Forms timer ran a full PBKDF2 derivation every 30 seconds to produce a sessionKey that nothing used. That timer fires on the interface thread, so the window stopped repainting and stopped accepting input for the length of every derivation. **Fixed:** the timer, its handler and the field were removed. Any future rotation must run off the interface thread and have a purpose.

Fixed in 3.1.2

Was	Where	Detail
The data files went wherever the process started.	`PasswordUtils.cs`, `MainForm.cs`, `LockoutManager.cs`	The three files were opened by bare name, which resolves against the working directory. From a terminal that was the current folder; from a double-click it could be an unwritable one, so the first save failed and the application looked hung. Two copies of the executable also had two vaults. **Fixed:** a new DataPaths class puts all three in %LOCALAPPDATA%\SecureVault, Program creates that folder first, and a vault left beside the executable by an earlier version is moved in on first start.
Debug symbols shipped with releases.	`SecureVault.csproj`	The release zip contained SecureVault.pdb alongside the executable. **Fixed:** release builds set DebugType to none.
The clipboard label was wrong.	`ViewPasswordForm.cs`	The screen said the clipboard clears in 10 seconds while the timer ran for 15. **Fixed:** one ClipboardClearSeconds constant now drives both the timer and the label.
Two windows were titled with their class names.	`ViewPasswordForm and ConfirmPasswordForm, Designer files`	The title bars read ViewPasswordForm and ConfirmPasswordForm. **Fixed:** View Entry and Confirm Master Password.

7. Parameters at a glance

Parameter	Value	Set in
Salt length	16 bytes, from RandomNumberGenerator	`PasswordUtils.GenerateSalt`
Key derivation	PBKDF2, HMAC-SHA256, 100,000 iterations by default (stored per vault), 32 bytes out	`PasswordUtils.DeriveEncryptionKey`
Login verifier	SHA-256 of the derived key, compared in constant time	`PasswordUtils.ComputeVerifier` `PasswordUtils.VerifyAndDeriveKey`
Vault encryption	AES-256, CBC mode, PKCS7 padding, random 16-byte IV per entry	`PasswordUtils.EncryptString`
Lockout	5 s after first failure, doubling, capped at 300 s, persisted	`LockoutManager`
Password policy	10+ chars, 1+ upper, 1+ lower, 1+ digit, 1+ symbol, common-substring block	`SecurityPolicy`
Reveal timer	10 seconds	`ViewPasswordForm`
Clipboard clear timer	15 seconds, conditional on contents unchanged	`ViewPasswordForm`
Data folder	%LOCALAPPDATA%\SecureVault	`DataPaths`
Framework	.NET 8, Windows Forms, C# 12; releases built without debug symbols	`SecureVault.csproj`

Prepared from the Version 3.1.2 source code as published in the repository. Where this document and the original coursework report differ, this document describes the code as it is.